Assignment Python Programming Fundamentals

1. Assignment Objectives

By completing this assignment, students will be able to:

- Analyze real-world problems and identify the information required to solve them.
- Convert problem statements into logical, step-by-step solutions.
- Develop interactive Python programs that accept input and produce meaningful output.
- Apply variables, data types, operators, strings, conditional statements, and loops in practical situations.
- Use counters, accumulators, and nested loops to solve problems involving repeated calculations and patterns.
- Test programs using different inputs and explain their results.

2. General Instructions

1. Read each scenario carefully before writing the program.
2. Develop a separate Python program for each scenario.
3. Identify the required inputs, processing steps, and expected outputs before coding.
4. Use meaningful variable names and appropriate comments.
5. Use clear output messages so that another person can understand the results.
6. Test each program with at least three different test cases, including a boundary case where appropriate.
7. Do not hard-code values that the user is expected to enter.
8. Use only the Python concepts covered in class. Do not use advanced features such as lists, dictionaries, user-defined functions, classes, file handling, or external libraries unless specifically permitted by the instructor.
9. If a scenario requires repeated input, use an appropriate loop rather than repeating the same statements manually.
10. Your programs must run without syntax errors and produce logically correct results.

Scenario 1: University Admission Eligibility Checker

A university receives applications from students seeking admission to an undergraduate program. The admission office needs a program to make a preliminary eligibility assessment.

The program should ask the applicant to enter their name, marks obtained in the intermediate examination, total possible marks, and entry-test score.

The university requires at least 60% intermediate marks and 50% entry-test marks for preliminary eligibility.

Your task:

- Calculate the applicant's intermediate percentage and entry-test percentage.
- Display the applicant's name, both percentages, and the preliminary eligibility result.
- Clearly explain which requirement was not satisfied if the applicant is ineligible.
- Handle cases where a score is negative or exceeds the corresponding maximum marks.

Scenario 2: Smart Electricity Bill Calculator

An electricity company calculates household bills according to monthly consumption. For this assignment, use the following simplified tariff:

Monthly units consumed	Rate per unit
First 100 units	Rs. 10
Next 100 units	Rs. 15
Units above 200	Rs. 20

The charges are progressive. For example, a household consuming 250 units pays Rs. 1,000 for the first 100 units, Rs. 1,500 for the next 100 units, and Rs. 1,000 for the remaining 50 units.

Your task:

- Ask the user for the consumer's name and monthly electricity consumption.
- Calculate the bill according to the progressive tariff.
- Display the consumer's name, units consumed, and total bill.
- Reject negative consumption values.
- Test the program with consumption below 100 units, exactly 100 units, exactly 200 units, and above 200 units.

Scenario 3: Password Strength and Validity Checker

A website requires users to create a password that meets some basic rules before registration.

For this assignment, a password is acceptable if it contains at least eight characters, includes at least one uppercase letter, includes at least one lowercase letter, and contains at least one digit.

Your task:

- Ask the user to enter a proposed password.
- Check its length and whether it contains the required character types.
- Display which requirements are satisfied and which are missing.
- Display whether the password meets all the specified rules.
- Allow the user to try again until an acceptable password is entered or the user chooses to stop.

Do not display the password's contents in the final report or submission screenshots. This is a basic classroom exercise, not a production-grade security checker.

Scenario 4: Student Cafeteria Order and Billing System

A university cafeteria offers the following menu:

Item	Price
Sandwich	Rs. 250
Burger	Rs. 400
Pizza slice	Rs. 300
Juice	Rs. 120

A customer can order one type of food at a time and specify the quantity. The cafeteria also offers a 10% discount if the food subtotal is at least Rs. 1,000.

Your task:

- Display the menu and prices.
- Ask the customer to enter their name, selected item, and quantity.
- Calculate the subtotal, applicable discount, and final bill.
- Display a clear receipt containing the order details.
- Handle an invalid menu selection and a non-positive quantity.
- Allow the customer to make another order until they choose to finish, keeping a running total of all orders.

Scenario 5: Personal Monthly Budget Assistant

A student receives a monthly allowance and wants to understand how much money remains after essential expenses.

The student records their monthly allowance and expenses for food, transport, internet, books, and other personal needs.

Your task:

- Ask for the student's name, monthly allowance, and each expense.
- Calculate total expenditure and remaining balance.
- Calculate the percentage of the allowance spent, where the allowance is positive.
- Classify the student's financial position as follows:
    - **Within Budget:** Total expenditure is less than the allowance.
    - **Budget Exhausted:** Total expenditure equals the allowance.
    - **Overspending:** Total expenditure exceeds the allowance.
- Display a warning if more than 80% of the allowance has been spent.
- Reject negative monetary values and handle a zero allowance safely.

Scenario 6: Public Transport Fare Estimator

A transport company charges passengers according to the distance travelled:

Distance	Fare
Up to 5 km	Rs. 50
More than 5 km and up to 15 km	Rs. 100
More than 15 km and up to 30 km	Rs. 180
More than 30 km	Rs. 250

Passengers aged 60 or above receive a 20% discount on the fare.

Your task:

- Ask for the passenger's name, age, and travel distance.
- Determine the applicable fare based on the distance.
- Apply the age-based discount when eligible.
- Display the base fare, discount amount, and final fare.
- Reject negative ages and non-positive travel distances.
- Test the program at the distance boundaries and for passengers on both sides of the age threshold.

Scenario 7: Online Shopping Checkout

An online store sells products at different prices. A customer may purchase several different products during one shopping session.

The store offers a 5% discount on a subtotal of Rs. 2,000 or more and a 10% discount on a subtotal of Rs. 5,000 or more. Only one discount is applied, using the highest eligible rate.

Your task:

- Ask for the customer's name.
- Repeatedly ask for a product name, price, and quantity.
- Calculate each item's cost and add it to the running subtotal.
- Stop accepting items when the customer chooses to finish.
- Apply the appropriate discount and calculate the final amount.
- Display the total number of items purchased, subtotal, discount rate, discount amount, and final bill.
- Reject invalid prices and quantities.

For this scenario, the number of items means the total quantity purchased, not the number of distinct product types.

Scenario 8: Library Late-Return Fine Calculator

A university library charges a fine when a book is returned after its due date. The fine is calculated as follows:

Days late	Fine per day
First 5 days	Rs. 10
Next 5 days	Rs. 20
Every day after the first 10 days	Rs. 50

The fine is progressive. For example, a book returned 12 days late incurs Rs. 50 for the first five days, Rs. 100 for the next five days, and Rs. 100 for the remaining two days.

Your task:

- Ask for the student's name, book title, and number of days late.
- Calculate the total fine using the progressive structure.
- Display the book title, days late, and total fine.
- If the book is returned on time or early, display that no fine is due.
- Reject a negative number of days late.

Scenario 9: Text Message Analyzer

A student is developing a simple text-analysis tool to understand the contents of a message.

Your task:

- Ask the user to enter a sentence or message.
- Display the original message, total number of characters, and message length excluding leading and trailing spaces.
- Count the number of vowels in the message.
- Count the number of digits and spaces.
- Check whether a specified word or phrase appears in the message, ignoring differences in letter case.
- Display the message in uppercase and lowercase.
- Do not use advanced text-processing libraries or built-in methods that directly solve the entire counting task.

For this exercise, vowels are a, e, i, o, and u. Count each occurrence, including repeated vowels.

Scenario 10: ATM Withdrawal Simulator

A bank wants a simple program to simulate cash withdrawals from a customer's account.

The customer enters their name, current account balance, and requested withdrawal amount. For this exercise, the ATM dispenses only multiples of Rs. 500.

Your task:

- Check whether the withdrawal amount is positive.
- Check whether it is a multiple of Rs. 500.
- Ensure that the withdrawal does not exceed the available balance.
- If all conditions are satisfied, deduct the amount from the balance.
- Display the transaction result and updated balance.
- Allow multiple transactions until the customer chooses to exit.
- Maintain a count of successful withdrawals and calculate the total amount withdrawn during the session.

Do not permit an unsuccessful transaction to change the account balance or successful-withdrawal statistics.

Scenario 11: Number Guessing Game

A teacher wants a simple guessing game for students. The program uses a fixed secret number between 1 and 100, selected by the programmer before the game begins.

The player repeatedly enters a guess until the correct number is found or the player reaches a maximum of seven attempts.

Your task:

- Ask the player to enter their name.
- Accept guesses and tell the player whether each guess is too high or too low.
- Count the number of attempts.
- Display a success message and the number of attempts if the player guesses correctly.
- If the player fails within seven attempts, reveal the secret number.
- Reject guesses outside the range of 1 to 100 without treating them as valid attempts.

The secret number must remain fixed during a game. You are not required to use random-number libraries.

Scenario 12: Employee Attendance and Salary Assessment

A company pays employees according to the number of days they attend work during a 26-working-day month. For this exercise, the employee receives a fixed daily wage, and there are no overtime payments or other deductions.

Your task:

- Ask for the employee's name, daily wage, and attendance for each of the 26 working days.
- Record attendance as P for present and A for absent.
- Reject invalid attendance entries and request a valid entry again for that day.
- Count the number of present and absent days.
- Calculate the employee's salary using the number of present days multiplied by the daily wage.
- Display the attendance summary, attendance percentage, and calculated salary.
- Warn the employee if attendance is below 80%.

Assume that the employee's daily wage is non-negative.

Scenario 13: Restaurant Table Reservation and Seating Plan

A restaurant has a rectangular seating arrangement consisting of rows and seats per row. The manager wants to generate a simple seating plan before customers arrive.

Your task:

- Ask for the number of rows and seats per row.
- Display each seat with a unique sequential seat number, starting from 1.
- Display the seating plan row by row.
- Calculate the total seating capacity.
- Ask how many customers have arrived and calculate the number of unoccupied seats.
- Reject negative dimensions and a customer count greater than the seating capacity.

For example, a restaurant with three rows and four seats per row should display 12 numbered seats arranged in three rows.

Scenario 14: Daily Temperature Monitoring

A weather station records the temperature each day and needs a simple report at the end of a monitoring period.

Your task:

- Ask the user how many days will be monitored.
- Accept the temperature for each day.
- Calculate the average temperature, highest temperature, and lowest temperature.
- Count how many days had temperatures above 35°C, below 10°C, and between 10°C and 35°C inclusive.
- Display the results in a clear report.
- Handle a monitoring period of zero days safely by requesting a valid positive number of days.

Scenario 15: Digital Savings Goal Tracker

A student wants to save money for a laptop costing Rs. 80,000. The student records the amount saved each week and wants to know when the goal will be achieved.

Your task:

- Ask for the student's name, savings goal, and initial savings.
- Accept the amount saved during each subsequent week.
- Maintain a running total of the savings.
- After each week, display the total savings and the remaining amount required to reach the goal.
- Stop when the goal is achieved or when the student chooses to stop.
- Count the number of weeks required to reach the goal.
- Display a final summary showing whether the target was achieved.
- Reject negative savings amounts and handle an initial balance that already meets or exceeds the goal.

Scenario 16: Cinema Ticket Booking and Revenue Report

A small cinema sells tickets at the following prices:

Customer category	Ticket price
Child (under 12)	Rs. 300
Adult (12–59)	Rs. 600
Senior (60 or above)	Rs. 400

The cinema wants to record ticket sales for a single screening.

Your task:

- Ask the operator to enter each customer's age.
- Calculate the ticket price according to the customer's category.
- Continue until the operator indicates that ticket sales are complete.
- Count the number of tickets sold in each category.
- Calculate total tickets sold and total revenue.
- Calculate the average ticket price if at least one ticket has been sold.
- Display a summary report of ticket sales and revenue.

For this exercise, assume each customer buys one ticket. Reject negative ages.

Scenario 17: Multiplication Practice and Performance Report

A school wants a practice program for students learning multiplication. The student selects a multiplication table and the number of questions to display.

Your task:

- Ask the student to enter their name, a table number, and the number of questions.
- Display the multiplication questions from 1 up to the selected number of questions.
- Ask the student to calculate each answer.
- Compare the student's answer with the correct result.
- Display feedback after each response.
- Count correct and incorrect answers.
- Calculate the student's percentage score and display a performance message at the end.

The program should reject invalid table numbers and non-positive question counts. Assume students enter integer answers.

Scenario 18: Simple Sales Performance Dashboard

A small shop wants to review the daily sales of its employees at the end of the working day. The shop manager records each employee's name and the total sales amount for that employee.

Your task:

- Ask the manager how many employees will be included in the report.
- For each employee, accept their name and total sales.
- Calculate the total sales of the shop and the average sales per employee.
- Identify the employee with the highest sales.
- Count how many employees achieved sales of at least Rs. 10,000.
- Display a report containing each employee's name and sales, total shop sales, average sales, highest-performing employee, and number of employees meeting the target.
- Reject negative sales amounts and handle a zero-employee report safely.

If two or more employees have the same highest sales, report that the highest sales are tied rather than incorrectly claiming a unique winner.

Submission Requirements

Submit the following items for all assigned scenarios:

1. **Python source files:** One .py file per scenario, with clear filenames such as scenario_01_admission.py.
2. **Problem analysis:** Briefly describe the problem, required inputs, processing steps, and expected outputs.
3. **Test cases:** Include at least three test cases per scenario, with the input values and expected results.
4. **Screenshots:** Capture screenshots showing the program running and its output for representative test cases. Include screenshots of important boundary cases or invalid inputs where applicable.
5. **Reflection:** Write a short paragraph describing the most challenging part of the assignment and how you solved it.

a. Organize all assignment files in a single folder. Include a README.md file listing all scenarios and their corresponding Python filenames.
b. Compress the folder into a .zip file and submit it via Google Classroom.
c. Name the ZIP file using the following format: **YOUR_ROLL_NUMBER_Python_asg_01.zip**
d. Before submitting, verify that the ZIP file opens correctly and contains all required files.

Warning: Corrupt, incomplete, or inaccessible ZIP files will receive **zero marks**.